Guia do autor de defeitos: tapSynth

Nov 2, 2026 · @Guilherme

Seu papel

Você cria os defeitos que o estudo vai injetar no tapSynth: 10 a 15 patches baseados em correções reais de plugins JUCE open source, mais algumas alterações inofensivas. Você trabalha sem acesso aos testes, de propósito.

O estudo compara abordagens automatizadas de verificação de plugins de áudio. Cada defeito é aplicado sozinho e todas as abordagens rodam contra ele. Se o autor dos defeitos conhecesse os testes, ou o autor dos testes conhecesse os defeitos, o resultado ficaria enviesado. Por isso os dois lados trabalham isolados.

O critério é um só: realismo. Crie defeitos que um desenvolvedor real poderia introduzir, como mostram as correções reais em que você se baseia. Não pense em quão fácil ou difícil seria detectá-los.

Regras de isolamento

Estas regras valem durante todo o trabalho. Se alguma não puder ser cumprida, pare e avise o responsável pelo estudo.

1. Trabalhe só neste repositório. Ele foi preparado sem testes e sem histórico git.
2. Não procure o repositório original do estudo nem outras cópias do tapSynth, no GitHub ou em outras pastas da máquina. Se encontrar por acaso testes, tolerâncias, áudios de referência ou configurações de validação do tapSynth, não leia: pare e avise.
3. Não escreva nem rode testes automatizados, validadores de plugin (como o pluginval) ou medições de áudio sobre os defeitos. A única verificação é: o patch aplica e o plugin compila.
4. Não escolha, descarte nem ajuste um defeito por ele ser fácil ou difícil de perceber. Ele deve seguir a correção real que o inspirou.
5. Nunca invente uma fonte. Todo link de correção real deve ter sido aberto por você e deve mostrar a correção descrita.
6. Entregue os patches só ao responsável pelo estudo. Não faça push para repositórios públicos.
7. Nunca escreva uma chave de API em nenhum arquivo. O plugin lê `GEMINI_API_KEY` do ambiente.
8. Toda mudança é revisada por uma pessoa antes do commit.

O sistema

O tapSynth é um sintetizador subtrativo em JUCE 8.0.9 (C++17, CMake), com 5 vozes, compilado como VST3 (e AU no macOS). A arquitetura está descrita no `CLAUDE.md` do repositório; leia as seções Build e Architecture antes de começar.

Caminho do áudio, por voz: osc1 + osc2 (seno, dente de serra ou quadrada, com FM e transposição) → ganho fixo → ADSR de amplitude → filtro de variáveis de estado, com cutoff modulado por um segundo ADSR e por um LFO. Depois da soma das vozes vêm um reverb e um medidor de nível.

Todo o estado do plugin está nos parâmetros do `apvts`, declarados em `createParams()`. O estado salvo pelo host é essa árvore serializada em XML. Um recurso de IA aplica parâmetros vindos de JSON por `applyParametersFromJson`.

Compilar sem testes (o repositório não os tem):

```
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Para defeitos no JUCE, clone o JUCE 8.0.9 (`git clone --branch 8.0.9 --depth 1 https://github.com/juce-framework/JUCE.git`) e compile com `-DTAPSYNTH_JUCE_DIR=/caminho/para/JUCE`.

Escopo

Defeitos entram em duas origens: o código do projeto e cinco classes de DSP do JUCE 8.0.9 que o tapSynth usa. As duas origens devem estar representadas.

Origem	Arquivo	O que contém
projeto	`Source/Data/OscData.cpp/.h`	Formas de onda, nota MIDI → frequência, transposição, FM, ganho do oscilador
projeto	`Source/SynthVoice.cpp/.h`	Início e fim de nota, soma dos osciladores, ganho fixo, ADSR, modulação e limite do cutoff, liberação da voz
projeto	`Source/Data/FilterData.cpp/.h`, `Source/Data/AdsrData.cpp/.h`	Tipo e parâmetros do filtro, LFO; parâmetros do ADSR
projeto	`Source/Data/MeterData.cpp/.h`	Medidor de nível (RMS e pico)
projeto	`Source/PluginProcessor.cpp/.h`	`createParams`, `prepareToPlay`, `processBlock`, reverb, salvar e restaurar estado, `applyParametersFromJson`

Origem	Arquivo	O que contém
JUCE	`modules/juce_audio_basics/utilities/juce_ADSR.h`	`juce::ADSR`
JUCE	`modules/juce_dsp/widgets/juce_Oscillator.h`	`juce::dsp::Oscillator`
JUCE	`modules/juce_dsp/processors/juce_StateVariableTPTFilter.h/.cpp`	`juce::dsp::StateVariableTPTFilter`
JUCE	`modules/juce_dsp/widgets/juce_Reverb.h`, `modules/juce_audio_basics/utilities/juce_Reverb.h`	`juce::dsp::Reverb` e o `juce::Reverb` que ele envolve
JUCE	`modules/juce_dsp/widgets/juce_Gain.h`	`juce::dsp::Gain`

Fora do escopo: `Source/UI/`, `Source/PluginEditor.*`, a chamada à API do Gemini, os formatos AU e AAX, `CMakeLists.txt`, os workflows e qualquer outro arquivo do JUCE.

Categorias de defeito

As seis categorias foram fixadas antes de qualquer defeito existir. Cada patch pertence a exatamente uma; use o identificador da primeira coluna no cabeçalho.

Identificador	Categoria	Exemplo
`altura`	Altura/frequência	Taxa de amostragem fixa em 44,1 kHz no cálculo do oscilador
`nivel`	Nível/ganho	Ganho de saída dobrado
`envelope`	Tempo/envelope	O release nunca chega ao fim
`filtro`	Filtro	Cutoff em unidade errada; limite do cutoff removido
`numerico`	Numérico	Filtro instável na ressonância máxima, gerando NaN
`estado`	Estado/host	Restauração de estado ignora um parâmetro; travamento com bloco de 1 amostra; alocação de memória no callback de áudio

Os exemplos mostram o tipo de defeito, não uma lista a reproduzir. Prefira o que as correções reais encontradas mostrarem.

Fontes dos defeitos

Cada defeito reproduz no tapSynth um bug que alguém corrigiu de verdade num plugin JUCE open source (ou no próprio JUCE). O patch reintroduz o bug, ou seja, é a correção ao contrário, adaptada ao código do tapSynth.

- Quantidade: 10 a 15 patches.
- Cobertura: pelo menos um patch por categoria, e as duas origens (projeto e JUCE) representadas.
- Onde procurar: commits e issues de plugins JUCE open source no GitHub, e o histórico do próprio JUCE (`git log` nos arquivos do escopo).
- Registro: para cada defeito, o link do commit ou da issue e uma frase sobre a correção original. Se a adaptação se afasta muito do bug original, diga como no cabeçalho.
- Um bug real que não tem equivalente no tapSynth fica de fora. Não force a adaptação.

Os mutantes automáticos (Mull) são outra fonte do estudo e não fazem parte deste trabalho.

Formato do patch

Cada defeito é um arquivo `experiment/faults/F<NN>-<nome-curto>.patch` (por exemplo `F03-release-sem-fim.patch`), com um cabeçalho em comentários seguido da saída de `git diff`. O `git apply` ignora o texto antes do primeiro `diff --git`.

```
# id: F03
# category: envelope
# origin: projeto
# target: Source/SynthVoice.cpp
# source: https://github.com/<dono>/<repo>/commit/<hash>
# original-fix: uma frase sobre o bug corrigido no projeto de origem
# adaptation: como o bug foi trazido para o tapSynth (uma ou duas frases)

diff --git a/Source/SynthVoice.cpp b/Source/SynthVoice.cpp
...
```

- `id`: `F01`, `F02` ... sem lacunas.
- `category`: um dos seis identificadores da tabela de categorias.
- `origin`: `projeto` ou `juce`.
- Um defeito por patch. O patch deve aplicar sozinho sobre o código limpo, sem depender de outro.
- Patches do projeto: caminhos relativos à raiz deste repositório (`Source/...`).

- Patches do JUCE: caminhos relativos à raiz do clone do JUCE 8.0.9 (`modules/...`), aplicáveis com `git -C /caminho/para/JUCE apply`.
- Mude o mínimo necessário para reproduzir o bug. Sem comentários no código que denunciem o defeito.

Alterações inofensivas

Além dos defeitos, crie 3 a 5 refatorações que não mudam o comportamento do plugin. Elas medem alarmes falsos. As outras alterações inofensivas do estudo (build Release em vez de Debug, outro sistema operacional, outro compilador) não são patches e não fazem parte deste trabalho.

- Arquivo: `experiment/harmless/H<NN>-<nome-curto>.patch`, mesmo formato dos defeitos.
- Cabeçalho: `id`, `kind: refatoracao`, `origin`, `target` e `description` (o que foi refatorado e por que o comportamento é o mesmo).
- Escopo: os mesmos arquivos da tabela de escopo.
- Critério: uma refatoração que um revisor aceitaria como típica de um projeto real e que não muda o comportamento. Exemplos: extrair uma função, renomear variáveis, reorganizar um laço, trocar um tipo de container equivalente.

Fluxo de trabalho

Para cada defeito ou alteração inofensiva, partindo sempre do código limpo:

1. Encontre a correção real, abra o link e confirme que ela mostra o bug descrito.
2. Localize o trecho equivalente no tapSynth ou no JUCE 8.0.9, dentro do escopo.
3. Edite o código para reintroduzir o bug.
4. Gere o patch com `git diff` (no JUCE, `git -C /caminho/para/JUCE diff`) e escreva o cabeçalho acima dele.
5. Volte ao código limpo (`git checkout -- .`) e confira que o patch aplica sozinho: `git apply --check experiment/faults/F<NN>-...patch`.
6. Aplique, compile o VST3 (`cmake --build build`) e confirme que compila sem erros.
7. Desfaça o patch e confirme que `git status` está limpo, exceto pelos arquivos em `experiment/`.

Um defeito que não compila não serve: o estudo mede o que os testes detectam num plugin que compila. Um travamento ou comportamento ruim em tempo de execução é válido.

Entrega

Entregue ao responsável pelo estudo as pastas `experiment/faults/` e `experiment/harmless/`, mais um `experiment/faults/INDEX.md` com uma linha por patch: id, categoria, origem, arquivo alvo e link da fonte. Os patches só entram no repositório do estudo depois que os testes forem congelados.

Antes de entregar:

- [ ] 10 a 15 defeitos, pelo menos um por categoria
- [ ] Defeitos nas duas origens (projeto e JUCE)
- [ ] 3 a 5 refatorações inofensivas
- [ ] Todo patch tem o cabeçalho completo
- [ ] Todo link de fonte foi aberto e mostra a correção descrita
- [ ] Todo patch aplica sozinho sobre o código limpo e compila
- [ ] Nenhum teste, validador ou medição de áudio foi usado
- [ ] Nenhuma chave de API em nenhum arquivo